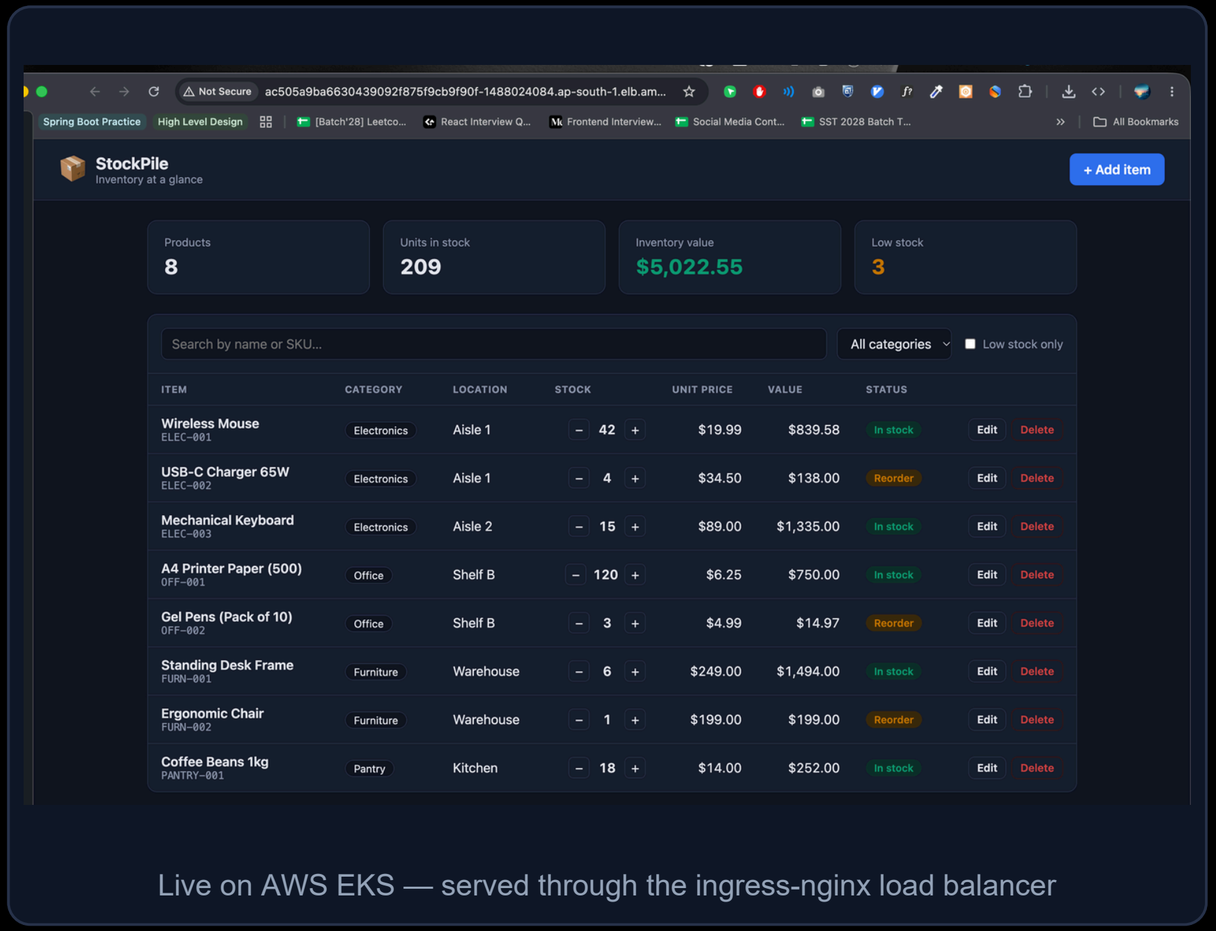
DEVSECOPS FINAL CAPSTONE

StockPile

A cloud-native inventory platform — from git push to a secured, autoscaled and monitored AWS EKS cluster





Live on AWS EKS — served through the ingress-nginx load balancer

Suja Rahaman
github.com/Shaikh-Suja-Rahaman/stockpile-DevOps-Project

What I built and why



Inventory management app

Track products, SKUs, stock and prices; adjust stock in one click; see low-stock items instantly.



Full delivery pipeline

Every commit is tested, scanned, packaged and rolled out to Kubernetes automatically.



Security built in, not bolted on

Vulnerability gates, non-root containers, Pod Security and network isolation.



Observable and self-scaling

Prometheus metrics, a Grafana dashboard and an HPA that scales the API under load.

7

REST API endpoints

24/24

automated tests passing

0

HIGH / CRITICAL CVEs

62

AWS resources via Terraform

The tools, layer by layer



Frontend

React 19 · Vite 7
nginx (unprivileged)



Backend

Node 22 · Express 5
Knex migrations



Database

PostgreSQL 16
StatefulSet + EBS gp3



Testing

Jest + Supertest
pg-mem test database



Containers

Multi-stage builds
Docker Compose



CI/CD & Security

GitHub Actions · Trivy
GHCR registry



Infrastructure

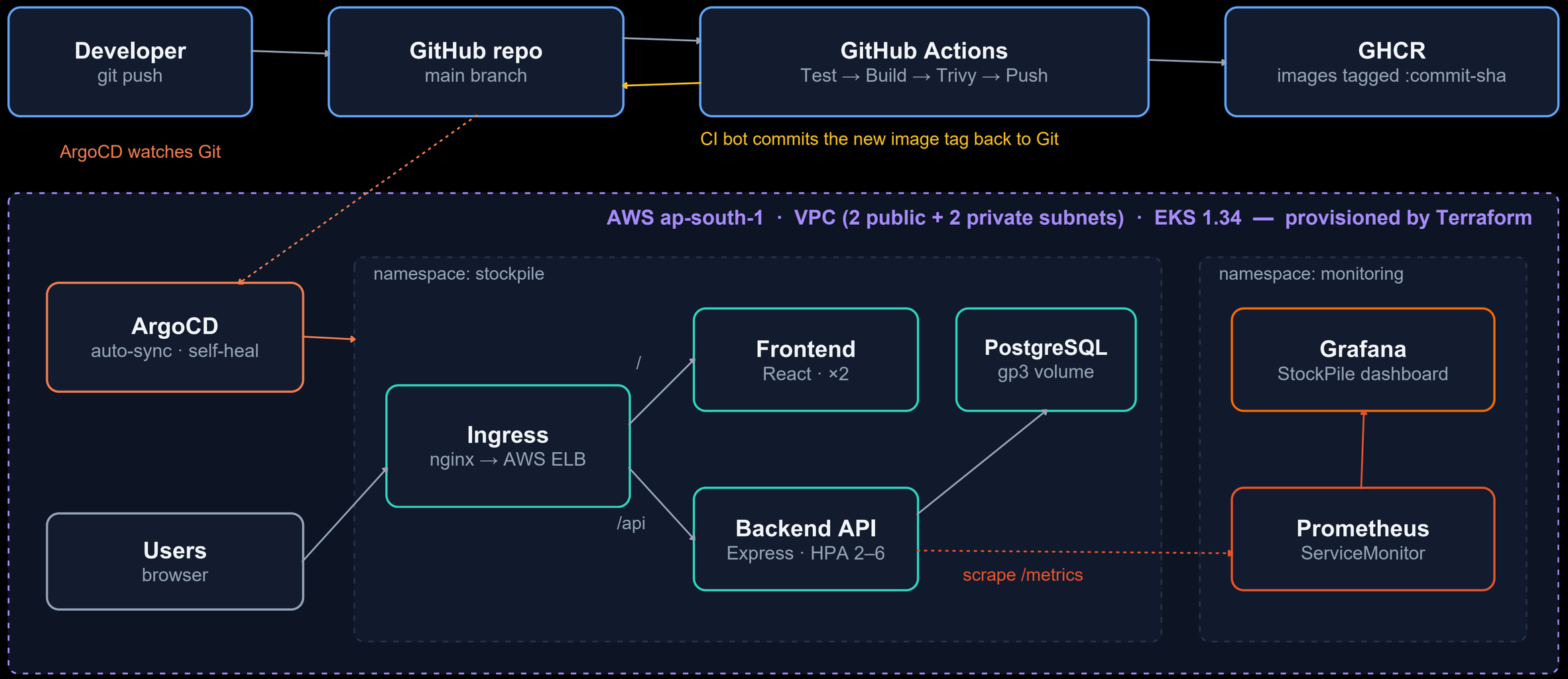
Terraform
AWS VPC + EKS 1.34



Delivery & Ops

Helm · ArgoCD · HPA
Prometheus · Grafana

From commit to cluster

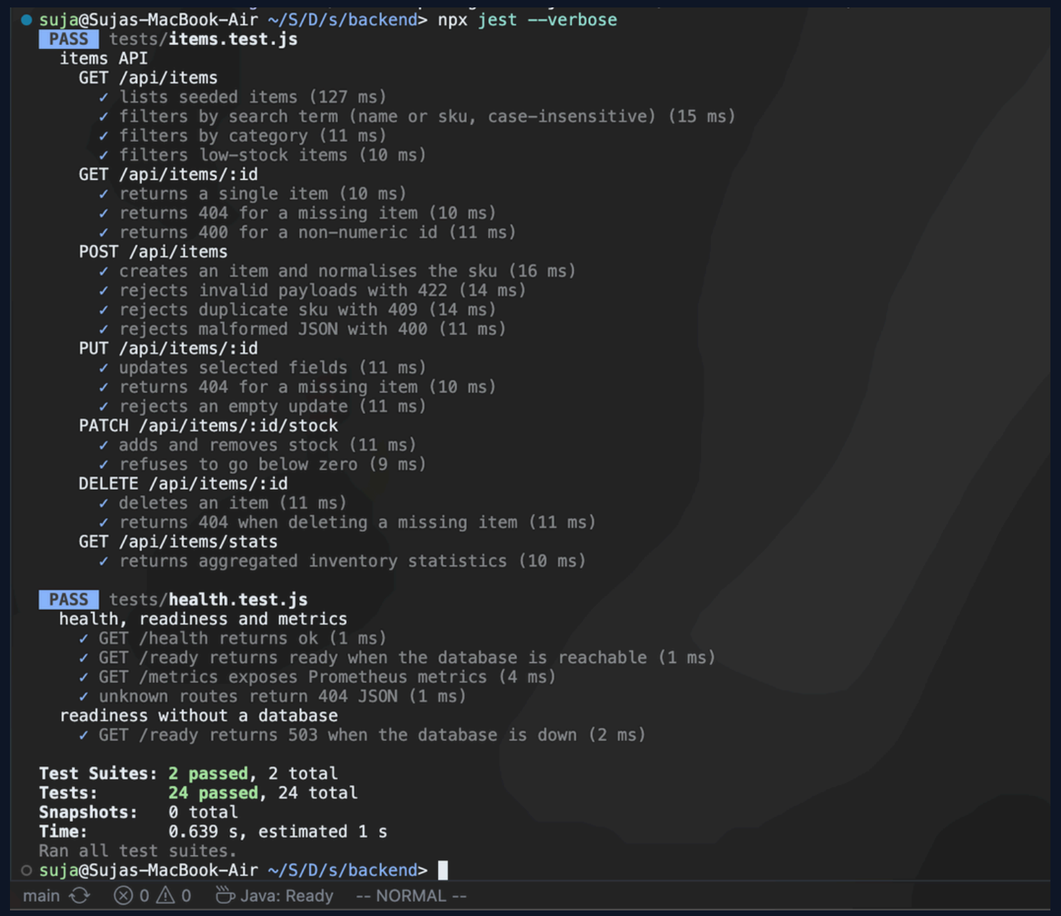


A small API, tested before anything ships

REST API

GET	/api/items	list · search · filter
GET	/api/items/stats	KPIs for the dashboard
POST	/api/items	create (unique SKU)
PUT	/api/items/:id	update
PATCH	/api/items/:id/stock	stock in / out
DELETE	/api/items/:id	remove
GET	/health · /ready · /metrics	probes + Prometheus

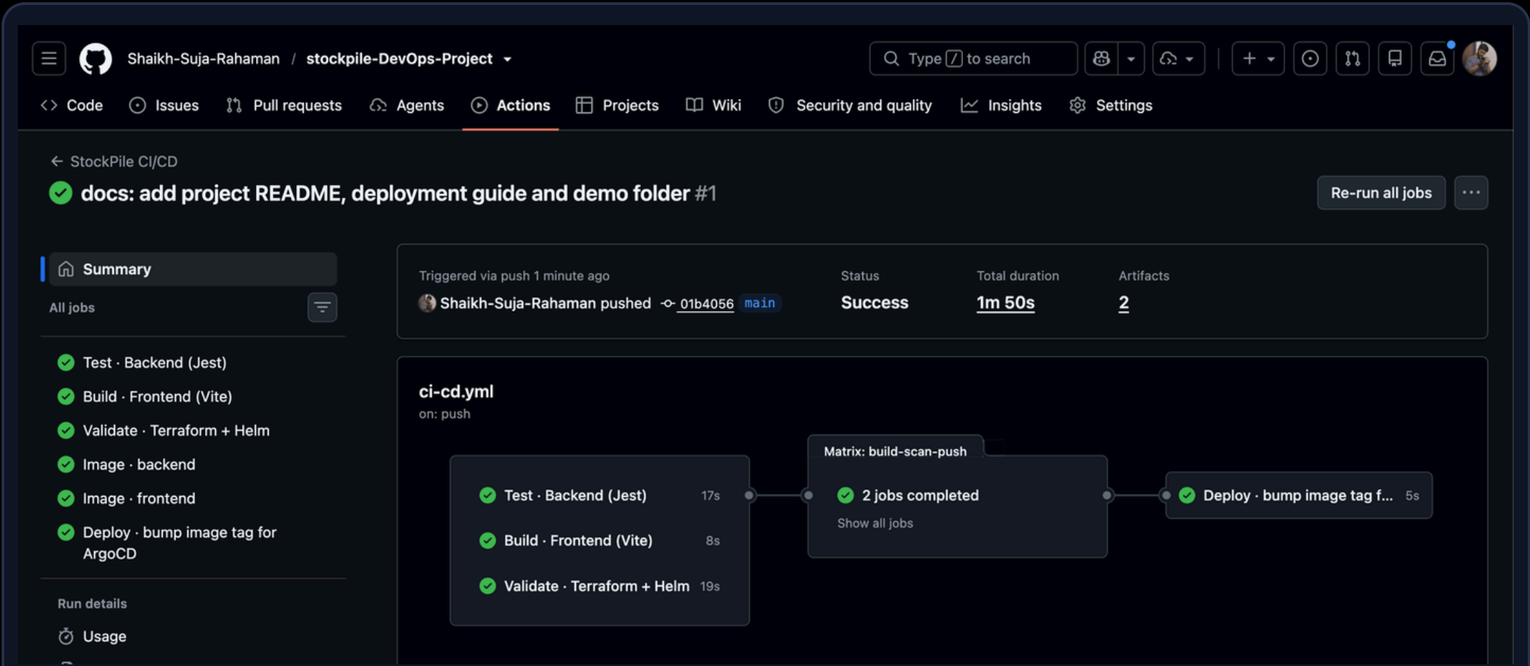
- **Knex migration** creates the items table (Alembic equivalent)
- **Isolated test DB** every test gets a fresh in-memory PostgreSQL
- **Readiness vs liveness** /ready = DB check, /health = process



24 / 24 Jest + Supertest tests passing

Every push to main runs five gated stages

1. **Test** — Jest 24/24 · npm audit
2. **Build** — Vite · Docker multi-stage
3. **Scan** — Trivy HIGH/CRITICAL gate
4. **Push** — GHCR :commit-sha
5. **GitOps** — bump tag → ArgoCD



Green pipeline run — all jobs passed in under 2 minutes

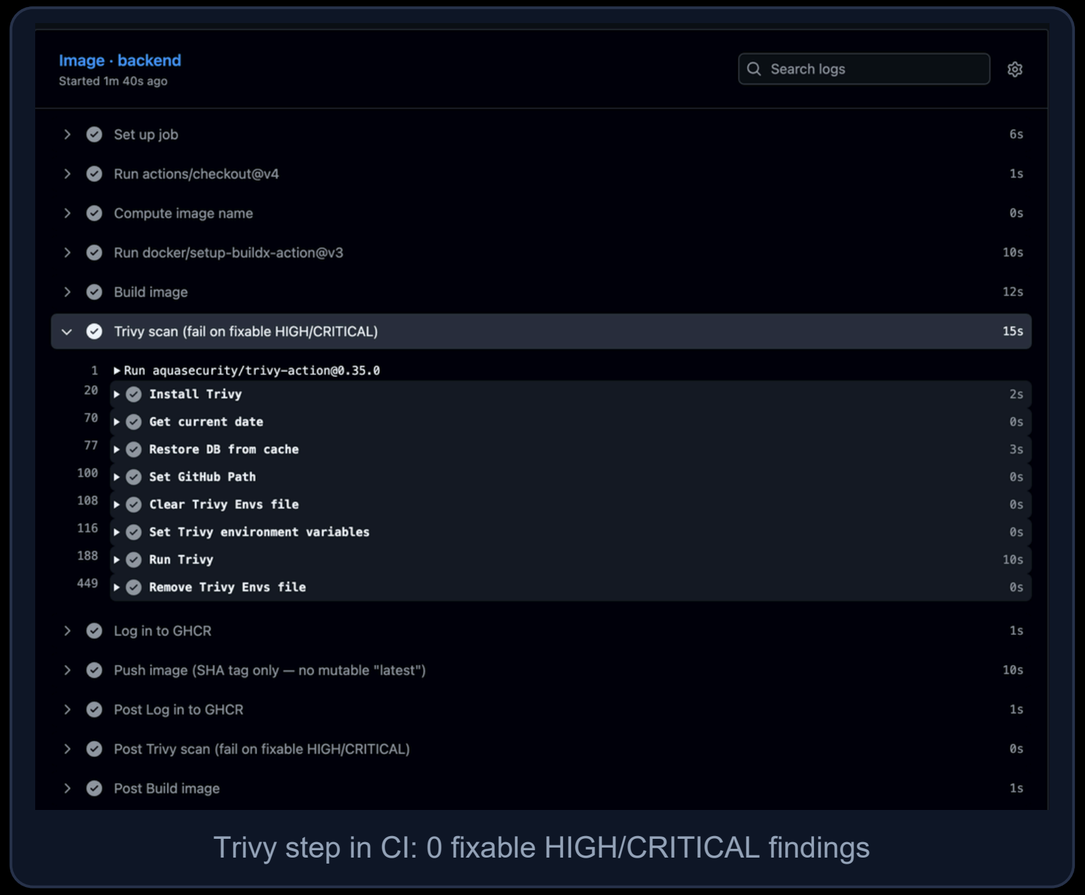
SHA tags, never "latest"
every pod traces back to the exact commit

PRs build + scan only
nothing unreviewed is pushed

No cluster credentials in CI
ArgoCD pulls; CI only commits a tag

Least-privilege tokens
write scopes only on the jobs that need them

Security gates at every layer



Trivy step in CI: 0 fixable HIGH/CRITICAL findings

Image scanning gate
Trivy fails the build on fixable HIGH/CRITICAL CVEs

Least-privilege containers
non-root · read-only root FS · all capabilities dropped

Pod Security "restricted"
enforced on the namespace by Kubernetes

NetworkPolicy
only backend pods can reach PostgreSQL

No secrets in Git
short-lived GITHUB_TOKEN; tfvars & state git-ignored

The gate caught real CVEs: nginx 1.28 base image → upgraded to 1.30 + apk upgrade → clean. Scanner pinned to trivy-action 0.35.0 after the March 2026 Trivy supply-chain attack.

Terraform builds the AWS network and EKS cluster

VPC	10.30.0.0/16 · ap-south-1 · 2 AZs
Subnets	2 public (load balancers) + 2 private (nodes) · 1 NAT gateway
EKS 1.34	managed node group · 2 × t3.large · Amazon Linux 2023
Add-ons	EBS CSI driver (IRSA) · VPC CNI with NetworkPolicy · CoreDNS

62 resources created by terraform apply — and all 62 removed by terraform destroy

```
module.eks.aws_eks_addon.this["aws-ebs-csi-driver"]: Creation complete after 45s [id=stockpile-eks:aws-ebs-csi-driver]

Apply complete! Resources: 62 added, 0 changed, 0 destroyed.

Outputs:

cluster_endpoint = "https://3B42A96A5422CC8705BB29A83D06C87D.gr7.ap-south-1.eks.amazonaws.com"
cluster_name = "stockpile-eks"
cluster_version = "1.34"
configure_kubectl = "aws eks update-kubeconfig --region ap-south-1 --name stockpile-eks"
private_subnets = [
  "subnet-09f7feb9d4ae1e8f9",
  "subnet-022bcf5f21c073f2e",
]
public_subnets = [
  "subnet-009587ea124709c77",
  "subnet-074f9a00a5786d3be",
]
region = "ap-south-1"
vpc_id = "vpc-02ddb6ae78870d679"
suja@Sujas-MacBook-Air ~/S/D/s/terraform (main)>
```

terraform apply — Apply complete! 62 added

```
suja@Sujas-MacBook-Air ~/S/D/stockpile (main)> aws eks describe-cluster --name stockpile-eks --region ap-south-1 --query 'cluster.[name,status,version,endpoint]' --output table && aws eks describe-nodegroup --cluster-name stockpile-eks --nodegroup-name (aws eks list-nodegroups --cluster-name stockpile-eks --region ap-south-1 --query 'nodegroups[0]' --output text) --region ap-south-1 --query 'nodegroup.[nodegroupName,status,scalingConfig.desiredSize,instanceTypes[0]]' --output table && kubectl get nodes -o wide
-------------------
```

One Helm chart packages the whole app

Deployments
frontend ×2 · backend ×2–6

StatefulSet
PostgreSQL + 2 Gi gp3 volume

Services
ClusterIP for every tier

Ingress
/ → frontend · /api → backend

HPA
scales backend on CPU

ServiceMonitor
Prometheus scrape config

NetworkPolicy
DB locked to backend

Probes
liveness /health · readiness /ready

```
suja@Sujas-MacBook-Air ~/S/D/stockpile (main)> kubectl get pods -n stockpile -o wide

NAME                                 READY   STATUS    RESTARTS   AGE   IP            NODE
GATES
stockpile-backend-6d698dbd5-2dc6z    1/1     Running   0          98s   10.30.2.70    ip-10-30-2-35.ap-
stockpile-backend-6d698dbd5-fzdt6    1/1     Running   0          84s   10.30.1.109   ip-10-30-1-5.ap-s
stockpile-frontend-7588d9c485-98tkh  1/1     Running   0          98s   10.30.2.28    ip-10-30-2-35.ap-
stockpile-frontend-7588d9c485-qvsll  1/1     Running   0          98s   10.30.1.178   ip-10-30-1-5.ap-s
stockpile-postgres-0                 1/1     Running   0          98s   10.30.1.156   ip-10-30-1-5.ap-s
suja@Sujas-MacBook-Air ~/S/D/stockpile (main)>
```

kubectl get pods — all Running across both nodes

```
suja@Sujas-MacBook-Air ~/S/D/stockpile (main)> helm list -n stockpile
NAME        NAMESPACE   REVISION  UPDATED                                 STATUS    CHART             APP VERSION
stockpile   stockpile   1         2026-10-16 19:41:13.214199 +0530 IST    deployed  stockpile-1.0.0   1.0.0
suja@Sujas-MacBook-Air ~/S/D/stockpile (main)>
```

helm list — release deployed

```
suja@Sujas-MacBook-Air ~/S/D/stockpile (main)> kubectl get ingress -n stockpile

NAME        CLASS   HOSTS   ADDRESS                                                                  PORTS   AGE
stockpile   nginx   *       ac505a9ba6630439092f875f9cb9f90f-1488024084.ap-south-1.elb.amazonaws.com  80      2m51s
suja@Sujas-MacBook-Air ~/S/D/stockpile (main)>
```

kubectl get ingress — address is the AWS ELB

values-dev.yaml local kind cluster **values-prod.yaml** EKS: GHCR images, gp3, HPA

Git is the single source of truth

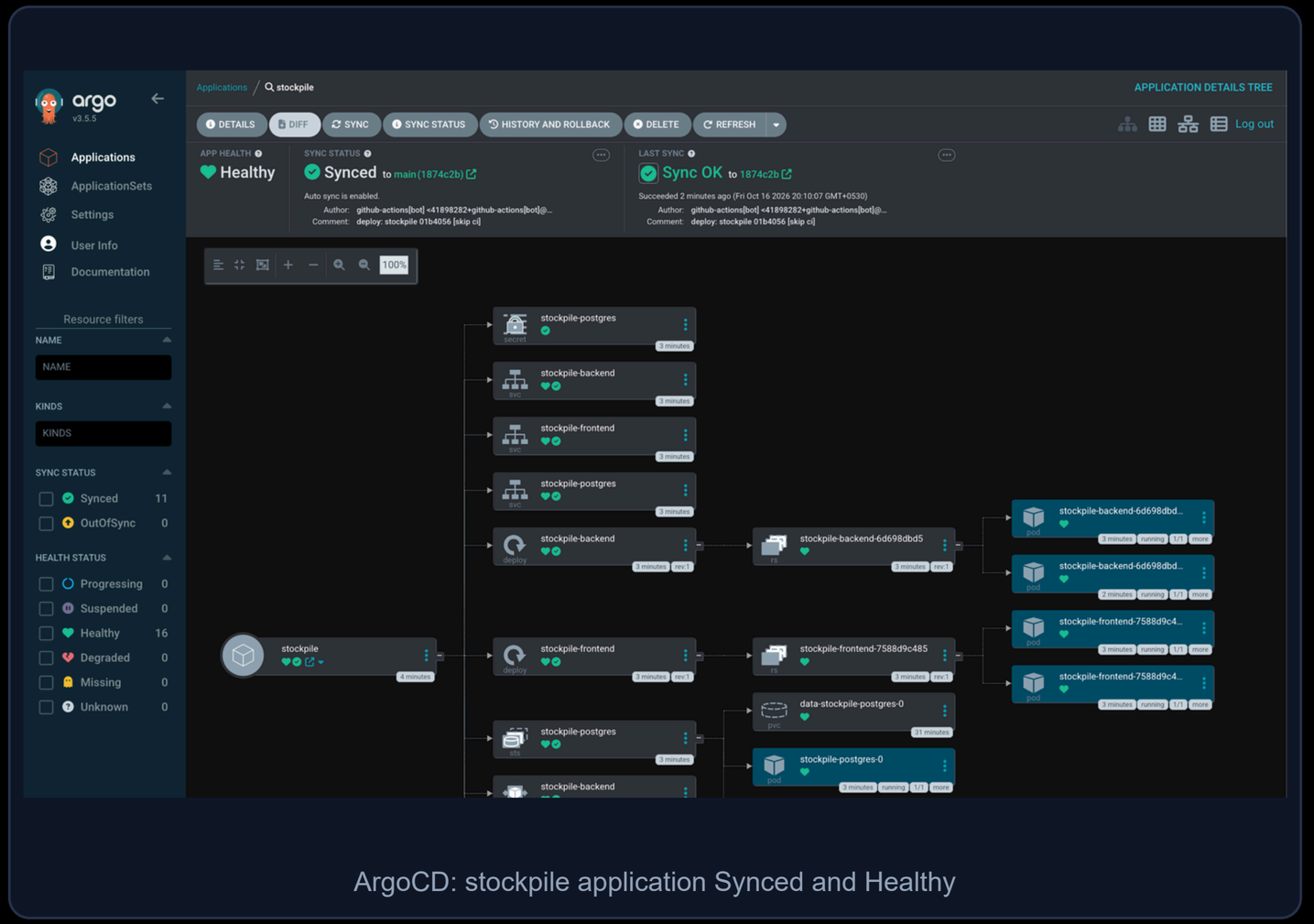
git push
a code change lands on main

CI tests, scans, pushes
image :sha published to GHCR

Bot commits the new tag
values-prod.yaml updated [skip ci]

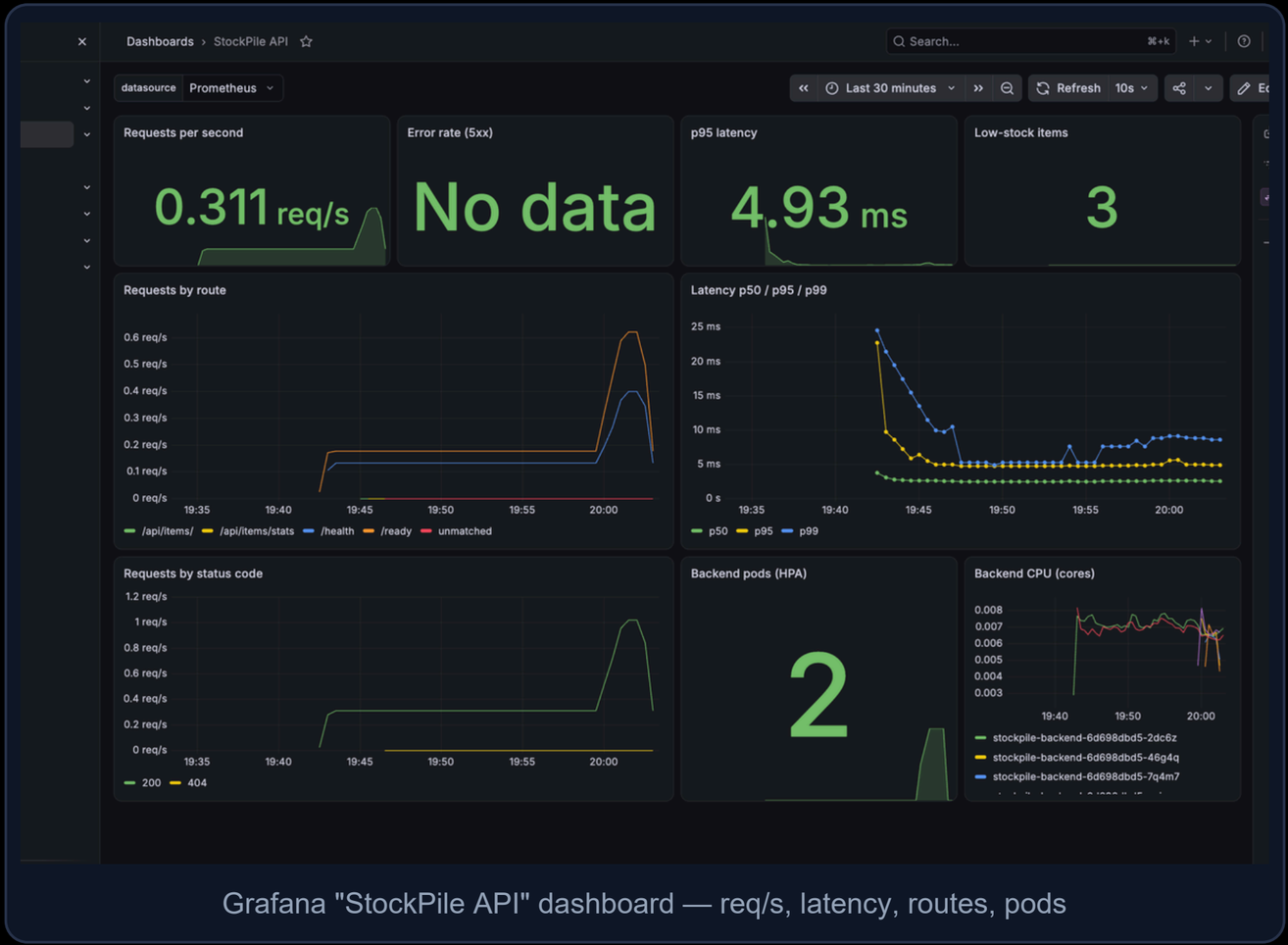
ArgoCD auto-syncs
rolling update · prune · self-heal

Self-heal demo: a manual kubectl scale is reverted by ArgoCD back to what Git says.

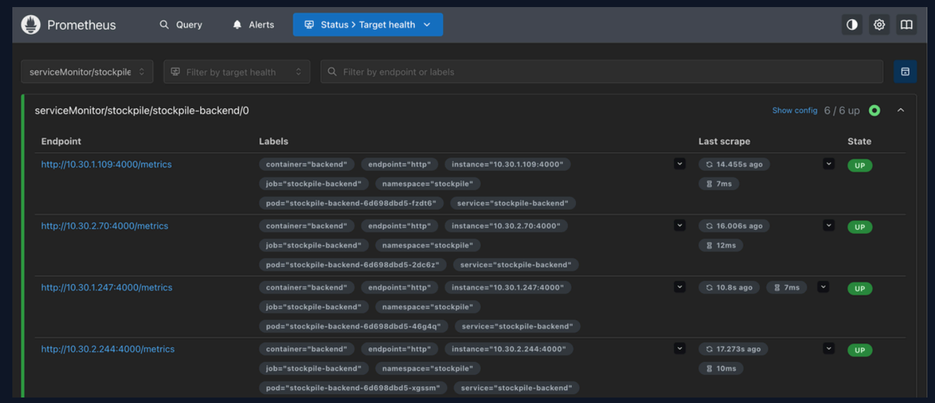


ArgoCD: stockpile application Synced and Healthy

Measured, watched and scaled automatically



Grafana "StockPile API" dashboard — req/s, latency, routes, pods



Prometheus: all backend targets UP

```
suja@Sujas-MacBook-Air ~/S/D/stockpile (main) [1]> kubectl get hpa -n stockpile

NAME                REFERENCE                      TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
stockpile-backend   Deployment/stockpile-backend   cpu: 7%/5%   2         6         6          19m
suja@Sujas-MacBook-Air ~/S/D/stockpile (main)>
```

HPA scaled the backend from 2 → 6 replicas under load

Custom metrics from /metrics

- http_requests_total
- request latency histogram
- stockpile_low_stock_items

Grafana dashboard ships as a ConfigMap — auto-imported, no manual clicks.

Breaking it on purpose, then cleaning up

Broken image tag

Symptom: ImagePullBackOff
Diagnosis: kubectl describe pod → events

Wrong Service selector

Symptom: Service with 0 endpoints
Diagnosis: compare selector vs pod labels

Bad readiness probe

Symptom: Running but 0/1 Ready
Diagnosis: probe path returns 404

```
module.eks.aws_security_group_rule.node["ingress_nodes_ephemeral"]: Destruction complete after 3s
module.eks.aws_iam_role.this[0]: Destruction complete after 1s
module.eks.aws_security_group_rule.node["ingress_cluster_8443_webhook"]: Destruction complete after 3s
module.eks.aws_security_group_rule.node["ingress_cluster_443"]: Destruction complete after 4s
module.eks.aws_security_group_rule.node["ingress_cluster_6443_webhook"]: Destruction complete after 5s
module.eks.aws_security_group_rule.node["ingress_cluster_9443_webhook"]: Destruction complete after 4s
module.eks.aws_security_group_rule.node["ingress_self_coredns_udp"]: Destruction complete after 5s
module.eks.aws_security_group_rule.node["ingress_cluster_kubelet"]: Destruction complete after 4s
module.eks.aws_security_group.cluster[0]: Destroying... [id=sg-0b87d7d62fb5e0b0d]
module.eks.aws_security_group.node[0]: Destroying... [id=sg-03277ac77b0b01602]
module.eks.aws_security_group.cluster[0]: Destruction complete after 1s
module.eks.aws_security_group.node[0]: Destruction complete after 1s
module.vpc.aws_vpc.this[0]: Destroying... [id=vpc-02ddb6ae78870d679]
module.vpc.aws_vpc.this[0]: Destruction complete after 1s

Destroy complete! Resources: 62 destroyed.
suja@Sujas-MacBook-Air ~/S/D/s/terraform (main)>
```

terraform destroy — all 62 resources removed

Teardown order matters

1. Delete the ArgoCD app + namespace (EBS volume)
2. Uninstall ingress-nginx (AWS load balancer)
3. Then terraform destroy

K8s-created load balancers would otherwise block VPC deletion.

What this project taught me

Scanners find real problems
Trivy blocked a vulnerable nginx base image — security gates are only useful when they can fail the build.

The supply chain is an attack surface
Even the scanner was compromised in 2026. Pin tool versions; pin actions to SHAs next.

Readiness ≠ liveness
Separate /health and /ready, plus an init container, made rollouts smooth with zero crash loops.

Storage needs plumbing on EKS
PersistentVolumes need the EBS CSI driver and a StorageClass — EKS 1.30+ ships no default.

Reproducible builds
Lockfiles must match the CI toolchain (Node 22 / npm 10), or npm ci fails in Docker.

GitOps simplifies delivery
CI never needs cluster credentials; Git history becomes the deployment history.

Thank you

Questions?

Repository
github.com/Shaikh-Suja-Rahaman/stockpile-DevOps-Project

Presented by
Suja Rahaman

Next steps for production

- Amazon RDS instead of in-cluster PostgreSQL
- TLS with ACM / cert-manager + Route 53 domain
- External Secrets / AWS Secrets Manager
- Pin GitHub Actions to commit SHAs
- Move from ingress-nginx to Gateway API / AWS LB Controller
- Remote Terraform state (S3 + locking)